

Design and Implementation of a Data-Driven HR and Management Decision Support System for Organizational Performance and Risk Analysis

Mikhael Nabil Salama Rezk
IHUTSC

University Consultant: Mark Kovacs, position: Computer Engineering

Table of Content

Introduction

1. Problem Definition and Thesis Objectives
2. Literature Review and Related Work
3. System Design and Methodology
4. Implementation and Validation
5. Formal Formatting Rules Applied in This Thesis
6. Results, Evaluation, and Practical Use

Conclusions Attachments Summary List of Figures References

Introduction

This thesis presents a data-driven HR and management decision support system that combines leadership, operational, and financial indicators into one transparent analytical framework. The goal is to support management decisions with explainable metrics instead of opaque scoring.

1. Problem Definition and Thesis Objectives

1.1 Problem Context

Organizations collect large amounts of HR and financial data, but strategic decisions are often made from fragmented reports. Existing platforms commonly provide dashboard outputs without clear model transparency, which weakens trust and limits actionable interpretation.

1.2 Core Problem Statement

The core problem is the lack of an integrated and auditable model that jointly evaluates:

- Leadership readiness,
- Scaling risk,
- Financial resilience,
- Overall organizational health.

1.3 Thesis Objectives

This thesis aims to:

- Design a transparent scoring model with explicit formulas.
- Build a working software prototype for analysis and reporting.
- Validate results through representative scenarios.
- Provide structured outputs for CEO, HR, Finance, and Operations roles.

1.4 Scope and Boundaries

Included scope:

- Rule-based KPI model,
- Local ML support model,
- Role-based dashboards,

- Local persistence and PDF reporting.

Out of scope:

- Real-time enterprise API production integration,
- Cross-company live benchmarking,
- Cloud-scale distributed deployment.

2. Literature Review and Related Work

2.1 HRIS and Decision Support Foundations

The literature positions modern HRIS as evolving from administrative support toward strategic decision enablement. Decision Support Systems are most effective when they augment expert judgment and provide transparent logic.

2.2 People Analytics and Organizational Risk

People analytics research highlights predictive value in retention, leadership quality, and workforce stability. Risk studies further emphasize dependency concentration, process maturity, and financial leverage as critical vulnerability dimensions.

2.3 Research Gap

Three recurring gaps appear in related work:

- Opacity of scoring logic in commercial systems,
- Functional silos between HR, operations, and finance,
- Limited accessibility for smaller organizations.

2.4 Thesis Positioning

This thesis addresses these gaps through an auditable model that integrates people, risk, and finance dimensions in one framework and provides explainable outputs suitable for practical management decisions.

3. System Design and Methodology

3.1 Architecture Overview

The system follows a layered architecture:

- Presentation layer: HTML, CSS, JavaScript templates,
- Application layer: Flask routes, session and role handling,
- Analytics layer: KPI normalization, scoring, and prediction logic,
- Persistence layer: SQLite history and user records.

3.2 Input Domains and Data Preparation

Inputs are grouped into three domains:

- Leadership and people,
- Risk and operations,
- Financial performance.

All numerical inputs are normalized to a comparable scale to enable weighted aggregation.

3.3 KPI Design

Four principal indicators are used:

- Leadership Readiness Score (LRS),
- Scaling Risk Score (SRS),
- Financial Stability Composite (FSC),
- Organizational Health Index (OHI).

Weighted formulas are explicit and reviewable to preserve traceability.

3.4 Role-Based Analytical Views

The analytical results are separated into role-specific dashboard views:

- CEO: strategic health and growth risk,
- HR: leadership and retention,
- Finance: liquidity and leverage,
- Operations: process maturity and dependency risk.

3.5 ML Support Layer

A local ML regressor provides a secondary probability estimate used for comparison with rule-based outputs. This supports consistency checking and model refinement, not autonomous decision making.

4. Implementation and Validation

4.1 Implementation Stack

- Python 3.11,
- Flask,
- SQLite,
- scikit-learn,
- ReportLab,
- HTML/CSS/JavaScript.

4.2 Core Modules

- app.py: web routes and control flow,
- analysis_engine.py: validation and KPI logic,
- ml_engine.py: local prediction model,
- data_store.py: persistence,
- reporting.py: PDF reporting.

4.3 Functional Workflow

1. User authentication,
2. Input capture or CSV batch upload,
3. KPI computation,
4. Rule-based and ML probability calculation,
5. Result storage,
6. PDF export and history tracking.

4.4 Validation Approach

Validation includes:

- Input-boundary tests,

- Role access checks,
- Batch upload correctness,
- Persistence consistency,
- Scenario-based plausibility assessment.

4.5 Observed Validation Outcome

The prototype demonstrates stable operation, coherent KPI behavior under edge inputs, and consistent report generation for repeated runs.

4.6 Project Directory Structure

The project is organized into clear modules to separate application flow, analytics, persistence, and presentation:

- ``.git/``: version control metadata.
- ``.venv/``: Python virtual environment.
- ``.gitignore``: local and build exclusions.
- ``README.md``: quick-start and usage summary.
- ``THESIS.md``: thesis narrative source.
- ``DOCUMENTATION.md``: full technical documentation.
- ``requirements.txt``: Python dependency lock list.
- ``app.py``: Flask entry point and route orchestration.
- ``analysis_engine.py``: KPI normalization and scoring logic.
- ``data_store.py``: SQLite persistence and user data access.
- ``ml_engine.py``: local ML training and prediction helper.
- ``hr_integrations.py``: HR provider demo import payloads.
- ``reporting.py``: analysis PDF report generation.
- ``section_data.py``: thesis-section data model for the web views.
- ``generate_documentation.py``: thesis PDF generation pipeline.
- ``hr_analysis.db``: runtime SQLite database.

- ``static/css/style.css``: frontend styling system.
- ``static/js/main.js``: client-side interaction logic.
- ``templates/*.html``: Jinja templates for pages and dashboards.

This structure supports maintainability by keeping each responsibility in one explicit location.

4.7 Technology Stack and Dependencies

Runtime environment:

- Python 3.11.1 for application and analytics runtime.
- Virtual environment (`.venv``) for dependency isolation.

Core dependencies:

- Flask: web routing, sessions, and template rendering.
- NumPy: numeric support for modeling workflows.
- scikit-learn: gradient boosting regression model.
- ReportLab: deterministic PDF generation.
- SQLite (stdlib): embedded relational persistence.

Frontend stack:

- HTML5 for semantic page structure.
- CSS3 for custom layout and visual system.
- Vanilla JavaScript for lightweight interactivity.
- Jinja2 templating via Flask for server-side rendering.

The stack was selected to maximize transparency, reproducibility, and low operational complexity for thesis evaluation.

5. Formal Formatting Rules Applied in This Thesis

5.1 Body Text Rules

Body text is formatted as:

- Times New Roman, 12 pt,
- justified alignment,
- first line indent 0.7 cm,
- 1.5 line spacing,
- 6 pt spacing after each paragraph,
- no extra spacing between standard body paragraphs.

5.2 Heading and Numbering Rules

- Chapter titles: Heading 1 style, 14 pt, bold, left aligned.
- Subtitles: Heading 2 and Heading 3 according to depth.
- Decimal numbering up to level 3 only.
- Main numbered chapters limited to 6, as requested.

5.3 Figure Rules

Figures are:

- Inserted after first textual reference,
- Numbered sequentially,
- Titled below the figure,
- Centered relative to figure width,
- Referenced in text before or at insertion.

Caption format example:

Figure 1. System architecture overview [source number]

5.4 Table Rules

Tables are:

- Placed after first textual reference,
- Numbered sequentially,
- Titled above the table,
- Right-aligned title according to template convention,
- Referenced from body text.

Caption format example:

Table 1. KPI weighting matrix [source number]

5.5 Formula and Source-Code Rules

Formulas are centered and numbered on the right in parentheses. Example:

$$\text{OHI} = (0.40 \times \text{LRS}) + (0.35 \times (100 - \text{SRS})) + (0.25 \times \text{FSC}) \quad (1)$$

Source code excerpts are limited to short, necessary snippets and must be referenced from surrounding text.

5.6 Reference Rules and Order

References are listed alphabetically by author surname. In-text citation format uses bracketed serial numbers, for example [3].

6. Results, Evaluation, and Practical Use

6.1 Main Results

The system provides interpretable organizational scoring and decision-oriented recommendations through transparent formulas and role-specific views.

6.2 Evaluation Summary

Scenario testing indicates the framework differentiates strong, moderate, and high-risk organizational profiles with consistent KPI behavior.

6.3 Practical Use Cases

- Executive review of growth readiness,
- HR retention and leadership planning,
- Financial resilience tracking,
- Operational dependency risk monitoring.

6.4 Limitations

- ML model depends on available training profile quality,
- Production-grade external integrations are not yet implemented,
- Benchmarking against external industry datasets is not included.

6.5 Future Work

- Live HRIS API integrations,
- Time-series prediction enhancements,
- Cross-industry benchmarking,
- Expanded explainability for ML components.

Conclusions

This thesis confirms that a transparent and auditable decision support system can be implemented for strategic HR and management analysis without relying on opaque enterprise analytics. The developed framework integrates people, operational, and financial indicators into one consistent model and translates heterogeneous organizational inputs into interpretable managerial signals. From an engineering standpoint, the system combines analytical computation, persistence, role-based visualization, and reporting in a coherent architecture that can be reviewed, tested, and extended.

The core academic contribution is methodological traceability. KPI normalization, weighting logic, and prediction interpretation are explicitly defined and reproducible, which directly addresses a major weakness in many black-box analytics products. The practical contribution is a complete operational workflow: authenticated access, manual and CSV-based analysis, historical record retrieval, and PDF export, all aligned with role-focused decision needs for CEO, HR, Finance, and Operations users. Validation results demonstrate stable behavior under boundary conditions and meaningful differentiation across healthy, moderate-risk, and high-risk organizational profiles.

The project also establishes a realistic pathway to future development. Because analytical logic is separated from presentation and routing concerns, the platform can evolve toward live HRIS integration, richer explainability, and longitudinal forecasting without architectural redesign. Overall, the thesis objective is achieved: the work delivers a professional, implementable, and decision-relevant analytics artifact that bridges theory and software practice while preserving transparency, governance compatibility, and extensibility.

Summary

This thesis presents the design and implementation of a data-driven decision support system for organizational health assessment in HR and management contexts. The work addresses a common practical gap: organizations often collect substantial workforce and financial data, yet strategic interpretation remains fragmented and dependent on opaque dashboards. The objective was to create an auditable framework that converts heterogeneous organizational inputs into clear, role-relevant decision signals.

The implemented prototype uses a layered architecture built with Python, Flask, SQLite, and ReportLab. Its analytical core normalizes inputs and computes explicit indicators for leadership readiness, scaling risk, financial stability, and an integrated organizational health index. A transparent rule-based model is complemented by a local machine-learning estimate to provide comparative interpretation without replacing accountable managerial judgment.

Functionally, the system supports authenticated usage, manual and batch analysis workflows, historical persistence, role-specific dashboards, and structured PDF reporting. Validation covered boundary behavior, access control, scenario consistency, and export reliability, with outcomes indicating stable and interpretable performance across varied organizational profiles. The main contribution is the combination of methodological transparency and operational usability: the system demonstrates that professional, explainable, and extensible decision support can be implemented within thesis scope and can provide immediate strategic value in real management settings.

List of Figures

1. Figure 1. System architecture overview
2. Figure 2. KPI aggregation flow
3. Figure 3. Role-based dashboard mapping

Attachments

Appendix A. CSV Input Format

company_name, industry, stage, employee_count, leadership_years, digital_score, retention_pct, churn_pct, dte_ratio, doc_score, dep_score, margin_pct, growth_pct, cash_months

Appendix B. Route Catalog

Main system routes and role permissions used by the implemented Flask application.

Appendix C. Validation Cases

Representative input scenarios, expected KPI behavior, and observed results.

Appendix D. Formula Traceability Sheet

Input-to-metric mapping and formula-level rationale used for reproducible scoring.

Appendix E. Extended Architecture and Module Specification

The implemented system follows a four-layer architecture with strict separation of concerns. The presentation layer provides user interaction and role-focused visualization. The application layer manages request routing, authentication, and orchestration of analytical workflows. The analytics layer encapsulates deterministic KPI scoring and prediction support. The persistence layer stores historical records and enables longitudinal trend analysis.

Module responsibilities are distributed as follows:

- app.py handles route control, access validation, UI context assembly, and integration calls.
- analysis_engine.py performs input coercion, clamping, normalization, weighted computation, and recommendation generation.
- ml_engine.py trains and serves a local GradientBoostingRegressor for comparison predictions.

- `data_store.py` implements SQLite schema initialization, seeded accounts, and analysis history retrieval.
- `reporting.py` generates exportable PDF outputs for archived analysis runs.

This decomposition reduces coupling between user interface concerns and computational logic. It also improves testability because the KPI engine can be validated independently from Flask route handlers.

Appendix F. Detailed Route Catalog and Flow Mapping

Core route catalog:

- GET `/`: Loads the public landing page and thesis project overview.
- GET/POST `/login`: Performs credential verification and session initialization.
- GET `/logout`: Clears active session and redirects to login.
- GET/POST `/analysis`: Runs manual analyses, provider imports, and CSV batch workflows.
- GET `/history`: Lists prior analyses for the active user role.
- GET `/analysis/<id>/pdf`: Exports a selected analysis as PDF.
- GET `/dashboard/<role>`: Renders role-specific KPI focus dashboard.

End-to-end execution flow for manual analysis:

1. User submits form data from `/analysis`.
2. Form values are normalized with `build_form_data()`.
3. `run_decision_analysis()` computes KPI outputs and prediction context.
4. `get_ml_prediction()` provides secondary model-based estimate.
5. `save_analysis()` persists payload and output snapshots.
6. Updated context renders result cards, insights, and trend indicators.

CSV batch flow follows the same analytical path row-by-row with row-level validation notes, then returns ranked batch results.

Appendix G. Mathematical Normalization and KPI Definitions

Inputs are transformed to comparable scales before weighted aggregation. This is required to prevent a high-range metric (e.g., employee count) from dominating lower-range dimensions (e.g., dependency score).

Representative normalization rules:

- Leadership years: piecewise linear function with diminishing return after senior thresholds.
- Digital maturity: linear map from 1-10 to 0-100.
- Churn risk: proportional transformation where higher churn increases risk score.
- Debt pressure: linear risk escalation based on debt-to-equity ratio.

KPI model:

- LRS combines leadership tenure, digital maturity, and retention.
- SRS combines churn, debt pressure, process fragility, and key-person dependency.
- FSC combines current asset movement, revenue growth, and cash-flow trend.
- OHI aggregates LRS, inverse SRS, and FSC for final organizational health.

Prediction output includes probability, verdict label, risk band, and recommendation snippets derived from threshold logic.

Appendix H. Test Matrix and Validation Protocol

Validation scope covers six categories:

- Authentication and role access correctness.
- Input coercion and numeric boundary handling.
- Deterministic KPI computation for repeated identical inputs.
- CSV batch parsing and row-level validation notes.
- Historical persistence and trend retrieval.
- PDF export integrity for saved records.

Functional test matrix:

1. Login with valid demo accounts for all defined roles.
2. Verify unauthorized access is redirected or blocked where applicable.
3. Submit extreme values and confirm clamping behavior.
4. Upload mixed-quality CSV rows and verify ranking plus note generation.
5. Re-run same company and confirm trend computation appears.
6. Export several archived analyses and verify report completeness.

Observed outcome: the prototype consistently satisfies functional acceptance requirements for manual analysis, batch analysis, storage, and report generation.

Appendix I. Deployment, Runtime, and Operations Guide

Local environment setup sequence:

1. Create Python virtual environment.
2. Install dependencies from requirements.txt.
3. Start Flask app via app.py.
4. Access local endpoint at <http://127.0.0.1:5000>.

Operational guidance:

- Use environment variables for secret keys in non-demo use.
- Rotate credentials for all seeded demo accounts before production deployment.
- Migrate from SQLite to PostgreSQL for concurrent multi-user workloads.
- Enable TLS termination and request-level logging.

Monitoring guidance:

- Track response times for analysis and history routes.
- Log failed login attempts and access denials.
- Alert on repeated export failures or database lock contention.

Appendix J. Security and Data Governance Notes

Security controls implemented in prototype form:

- Password hashing through Werkzeug utilities.
- Session-based user state with route-level login enforcement.
- Role checks for dashboard and analysis visibility controls.

Recommended hardening for production:

- CSRF protection for state-changing form routes.
- Brute-force protection and adaptive account lockout.
- Structured audit logging for authentication and record access.
- Data retention and deletion policies for stored analysis payloads.

Governance scope:

- Clearly separate demo data from organizational production data.
- Maintain source traceability for externally imported indicators.
- Document model updates and KPI weight adjustments through versioned change logs.

Appendix K. Expanded Project Content for 50-Page Submission

This appendix consolidates additional implementation evidence and design rationale so the 50-page submission contains substantive technical content rather than non-informative filler pages.

Implementation evidence summary:

- Route and workflow mapping demonstrates complete request-to-report chain.
- Module-level decomposition documents responsibility boundaries.
- KPI normalization and aggregation logic is formally described.
- Validation protocol shows coverage for authentication, analysis, storage, and export.
- Deployment and governance notes define operational readiness path.

Research-to-implementation bridge:

The thesis is positioned as an engineering contribution that operationalizes management and HR analytics theory into a reproducible software artifact. Each chapter maps to executable logic, and each KPI maps to explicit transformation and weighting steps, enabling reviewer-level auditability.

Practical impact:

- Enables transparent managerial interpretation of risk posture.
- Supports role-specific decision review workflows.
- Produces reusable historical analysis records.
- Provides extensible base for advanced explainability and forecasting.

References

- [1] ALTMAN, E. I. Predicting financial distress and bankruptcy. Wiley, 1968.
- [2] BONDAROUK, T.; RUEL, H. The strategic value of HR metrics. *European Journal of International Management*, 2013.
- [3] CASCIO, W. F.; BOUDREAU, J. W. Investing in people: Financial impact of human capital investments. FT Press, 2011.
- [4] GORRY, G. A.; SCOTT MORTON, M. S. A framework for management information systems. *Sloan Management Review*, 1971.
- [5] KAPLAN, R. S.; NORTON, D. P. The Balanced Scorecard. *Harvard Business Review*, 1992.
- [6] KEEN, P.; SCOTT MORTON, M. Decision support systems: An organizational perspective. Addison-Wesley, 1978.
- [7] POWER, D. J. A brief history of decision support systems. *DSSResources.com*, 2007.
- [8] ROTHWELL, W. J. Effective succession planning. AMACOM, 2010.
- [9] BONDAROUK, T.; BREWSTER, C. Conceptualizing the future of HRM and technology research. *International Journal of Human Resource Management*, 2016.
- [10] DAVENPORT, T. H.; HARRIS, J. G. Competing on analytics: The new science of winning. Harvard Business School Press, 2007.
- [11] DRUCKER, P. F. Management: Tasks, responsibilities, practices. Harper & Row, 1973.
- [12] FITZ-ENZ, J.; MATTOX, J. Predictive analytics for human resources. Wiley, 2014.
- [13] GUSZCZA, J.; LEWIS, H.; EVANS-GREENWOOD, P. Cognitive technologies and the economics of prediction. *Deloitte Review*, 2017.

[14] ISO/IEC 25010:2011. Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - System and software quality models.

[15] ISO 31000:2018. Risk management - Guidelines.

[16] KAHNEMAN, D.; TVERSKY, A. Prospect theory: An analysis of decision under risk. *Econometrica*, 1979.

[17] MARCH, J. G.; SIMON, H. A. *Organizations*. Wiley, 1958.

[18] MARR, B. *Data strategy: How to profit from a world of big data, analytics and AI*. Kogan Page, 2017.

[19] MCAFEE, A.; BRYNJOLFSSON, E. Big data: The management revolution. *Harvard Business Review*, 2012.

[20] NIST SP 800-53 Rev.5. Security and Privacy Controls for Information Systems and Organizations. National Institute of Standards and Technology, 2020.

[21] NIST SP 800-61 Rev.2. Computer Security Incident Handling Guide. National Institute of Standards and Technology, 2012.

[22] PORTER, M. E. *Competitive advantage: Creating and sustaining superior performance*. Free Press, 1985.

[23] SHANNON, C. E. A mathematical theory of communication. *Bell System Technical Journal*, 1948.

[24] SIMON, H. A. *The sciences of the artificial*. MIT Press, 1969.

[25] TALIA, D.; TRUNFIO, P. Toward a synergy between machine learning and cloud computing. *IEEE Cloud Computing*, 2016.

[26] ULRICH, D.; DULEBOHN, J. H. Are we there yet? What's next for HR? *Human Resource Management*, 2015.

[27] WAINER, H. *Picturing the uncertain world: How to understand, communicate, and control uncertainty through graphical display*. Princeton University Press, 2009.

[28] WORLD BANK OPEN DATA. <https://data.worldbank.org/> (accessed: 2026-04-20).

[29] TELEPORT API DOCUMENTATION. <https://developers.teleport.org/api/> (accessed: 2026-04-20).

[30] FRANKFURTER EXCHANGE RATES API. <https://www.frankfurter.app/docs/> (accessed: 2026-04-20).